

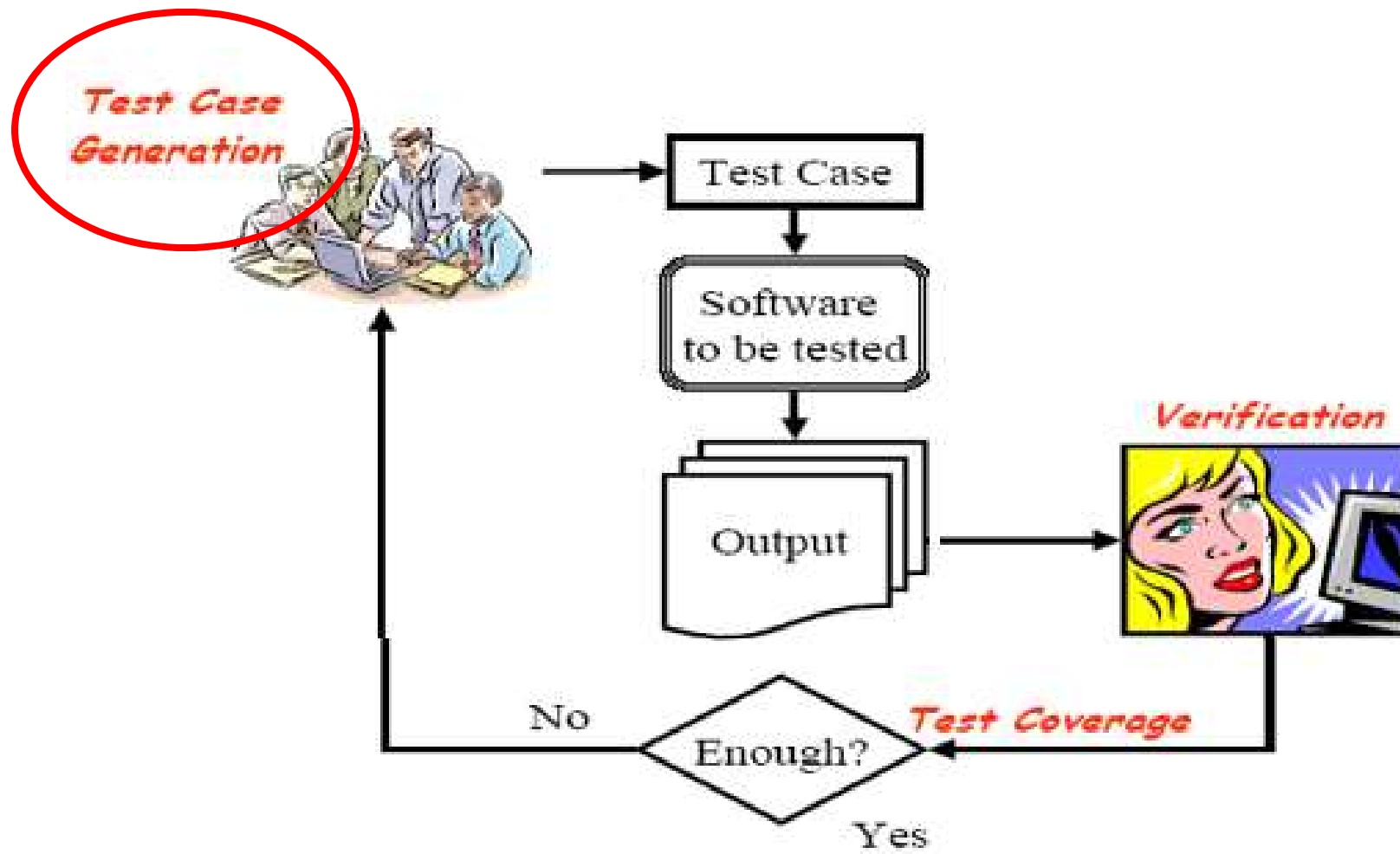
# 黑盒测试及测试用例生成

---

东南大学计算机科学与工程学院

李必信

# Testing process



# 目录

---

- 一 黑盒测试概述
- 二 黑盒测试用例设计和生成方法
  - 2.1 等价类划分法
  - 2.2.因果图(判定表)法
  - 2.3 边界值分析法

# 一 黑盒测试概述

---

1. 如何深入理解黑盒测试？
2. 黑盒测试有什么特点？
3. 如何实施黑盒测试？

# Testing Tactics



- Tests based on *spec*
- Test covers as much *specified* behavior as possible

- Tests based on *code*
- Test covers as much *implemented* behavior as possible

# 1. 如何理解黑盒测试？

---

- 1. 有时无法获取程序代码
- 2. 可以发现其它测试遗漏的**逻辑缺陷**（如，因果关系等）
- 3. 尽早进行黑盒测试可以尽早发现软件功能缺陷
- 4. 适用于各个测试阶段：单元测试，集成测试，系统测试，回归测试

# 1. 如何理解黑盒测试…

---

## ■ 定义

黑盒测试:一种基于规格说明（Spec），不要求考察代码，以用户视角进行的测试

其它称谓: 功能测试、基于规格说明的测试

## ■ 意义

黑盒测试有助于软件产品的总体功能验证:

1. 检查明确需求和隐含需求
2. 采用有效输入和无效输入
3. 包含用户视角

# 1. 如何理解黑盒测试…

---

## ■ 实施者

专门的软件测试部门；有经验的测试人员

## ■ 步骤

- (1) 熟悉规格说明书，理解测试需求；
- (2) 生成测试用例；
- (3) 执行测试；
- (4) 判定测试结果



## 2. 黑盒测试有什么特点？

---

- 黑盒测试特点：一种基于需求的测试

- 目的

- 确认（V&V）软件需求规格说明书列出的需求是否都正确实现

- 前提

- （1）软件需求规格说明书已经过仔细评审

- （2）隐含需求已经明确化

# 基于需求的测试.....

---

举例：

锁和钥匙系统的需求



# 基于需求的测试.....

## ■ 需求规格说明样本

锁和钥匙系统的需求规格说明（Spec）

| 序号 | 需求标识  | 需求描述                   | 优先级 |
|----|-------|------------------------|-----|
| 1  | BR-01 | 插入号码为123的钥匙并顺时针转动，应能上锁 | 高   |
| 2  | BR-02 | 插入号码为123的钥匙并逆时针转动，应能开锁 | 高   |
| 3  | BR-03 | 只有号码为123的钥匙可以用来上锁和开锁   | 高   |
| 4  | BR-04 | 其它东西都不能用来上锁            | 中   |
| 5  | BR-05 | 其它东西都不能用来开锁            | 中   |
| 6  | BR-06 | 锁受重物撞击也不能被打开           | 中   |
| 7  | BR-07 | 锁和钥匙重量必须为150g左右        | 低   |
| 8  | BR-08 | 开锁和上锁转动方向可变，便于左手人士使用   | 低   |

# 基于需求的测试.....

## ■ 需求跟踪矩阵（RTM）

需求跟踪矩阵样本

| 需求标识  | 需求描述                   | 优先级 | 测试条件          | 用例标识                 | 测试阶段 |
|-------|------------------------|-----|---------------|----------------------|------|
| BR-01 | 插入号码为123的钥匙并顺时针转动，应能上锁 | 高   | 使用号码为123的钥匙   | LOCK_001             | 单元组件 |
| BR-02 | 插入号码为123的钥匙并逆时针转动，应能开锁 | 高   | 使用号码为123的钥匙   | LOCK_002             | 单元组件 |
| BR-04 | 其它东西都不能用来上锁            | 中   | 使用IC卡<br>使用发卡 | LOCK_005<br>LOCK_006 | 集成   |
| BR-06 | 锁受重物撞击也不能被打开           | 中   | 使用石头砸锁        | LOCK_011             | 系统   |
| BR-08 | 开锁和上锁转动方向可变，便于左手人士使用   | 低   |               |                      | 未实现  |

# 基于需求的测试.....

## ■需求跟踪矩阵（RTM）

测试执行数据样本

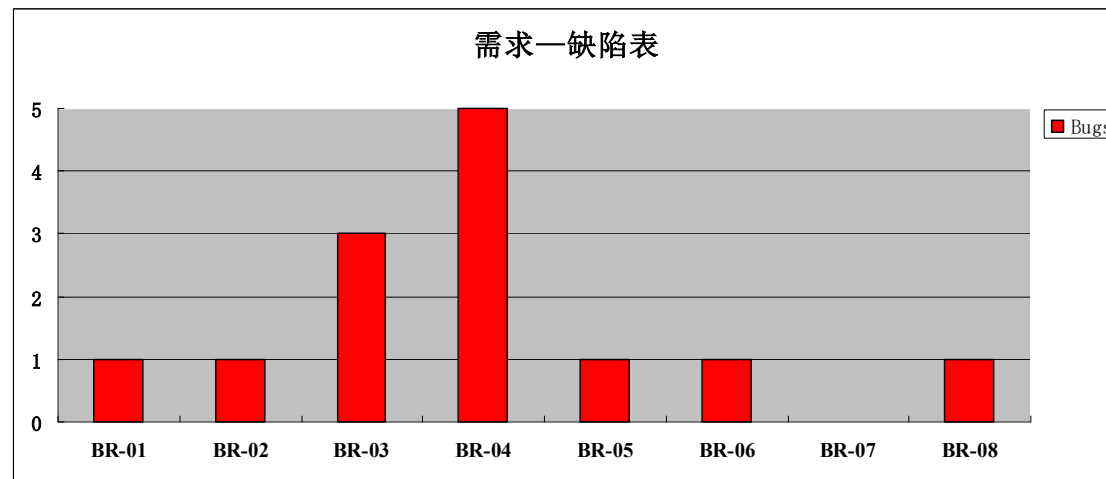
| 序号  | 需求标识  | 优先级 | 测试用例             | 用例总数 | 通过用例 | 未通过用例 | 通过率 | 缺陷数 |
|-----|-------|-----|------------------|------|------|-------|-----|-----|
| 1   | BR-01 | 高   | Lock_001         | 1    | 1    | 0     | 100 | 0   |
| 2   | BR-02 | 高   | Lock_002         | 1    | 1    | 0     | 100 | 0   |
| 3   | BR-03 | 高   | Lock_003,004     | 2    | 1    | 1     | 50  | 3   |
| 4   | BR-04 | 中   | Lock_005,006,007 | 3    | 2    | 1     | 67  | 5   |
| ... | ...   | ... | ...              | ...  | ...  | ...   | ... | ... |
| 合计  | 8     |     |                  | 12   | 10   | 2     | 83  | 12  |

# 基于需求的测试.....

## ■ 需求跟踪矩阵（RTM）

1. 测试用例已通过率83%； 已满足需求71%;
2. 有1个高优先级需求BR-03没有通过测试，对应3个缺陷；
3. 有1个中优先级需求BR-04没有通过测试，对应5个缺陷；
- .....

形成图表：



# 基于需求的测试.....

---

## ■ 优点

- ① 黑盒测试与软件具体实现无关，所以如果软件实现发生了变化，测试用例仍然可以使用；
- ② 设计黑盒测试用例可以和软件实现同时进行，因此可以压缩项目总的开发时间。

## 需求跟踪矩阵

## REQUIREMENTS TRACEABILITY MATRIX

项目名称  
Project Title: \_\_\_\_\_编制日期  
Date Prepared: \_\_\_\_\_

| 用户需求项标号 | 用户需求标题   | 用户需求变更标识 | 软件需求功能标号 | 软件需求功能标题   | 软件需求变更标识 | 需求状态 | 变更序号 | 当前状态 | 概要设计状态 | 对应概要设计章节 | 详细设计状态 | 对应详细设计章节 | 单元测试用例 | 集成测试用例 | 系统测试用例 | 对应代码       | 系统编码状态 |
|---------|----------|----------|----------|------------|----------|------|------|------|--------|----------|--------|----------|--------|--------|--------|------------|--------|
| FR001   | 合同网上审批流程 | 增加       | 3.1      | 合同评审       | 增加       | 已批准  |      | 需求开发 |        |          |        |          |        |        |        |            |        |
|         |          |          | 3.1.1    | 网上审批流程     | 增加       | 已批准  |      | 结项   | 编写     | 4.1.2.3  | 评审通过   | 6.1.2    | E1     | E5     | T3.1   | emis导出业务代码 | 已单元测试  |
|         |          |          | 3.1.2    | 打印合同评审表    | 增加       | 已批准  |      | 结项   | 评审通过   | 4.1.2.4  | 评审通过   | 6.1.3    | E2     | E6     | T3.2   | emis导出业务代码 | 已单元测试  |
|         |          |          | 3.1.3    | 脚本控制字段     | 增加       | 已批准  |      | 系统设计 | 评审通过   | 4.1.2.5  | 评审通过   | 6.1.4    | E3     | E7     | T3.3   | emis导出业务代码 | 已单元测试  |
|         |          |          | 3.2      | 合同评审查询     | 增加       | 未批准  |      |      |        |          |        |          |        |        |        |            |        |
|         |          |          | 3.2.1    | 合同评审结果综合查询 | 增加       | 已批准  |      | 结项   | 评审通过   | 4.1.2.6  | 评审通过   | 6.1.5    | E4     | E8     | T3.4   | emis导出业务代码 | 已单元测试  |
| FR002   | 合同综合查询   | 原始       | 3.3.1    | 综合查询       | 原始       | 已批准  |      | 结项   | 评审通过   | 4.1.2.7  | 评审通过   | 6.1.6    | E5     | E9     | T3.5   | emis导出业务代码 | 已单元测试  |
|         |          |          | 3.3.2    | 能按树表结构查询   | 原始       | 已批准  |      | 结项   | 评审通过   | 4.1.2.8  | 评审通过   | 6.1.7    | E6     | E10    | T3.6   | emis导出业务代码 | 已单元测试  |
|         |          |          | 3.3.4    | 打印功能       | 原始       | 已批准  |      | 结项   | 评审通过   | 4.1.2.10 | 评审通过   | 6.1.9    | E8     | E12    | T3.8   | emis导出业务代码 | 已单元测试  |



### 3. 如何实施黑盒测试？

---

- 黑盒测试实施： 正面测试和负面测试

# 正面测试

---

正面测试和正面测试用例：

通过正面测试用例产生一组预期输出验证产品需求

目的：

证明软件对于每条规格说明和期望都能通过

# 正面测试.....

## 正面测试用例举例

| 需求标识  | 输入1       | 输入2   | 当前状态 | 预期状态 |
|-------|-----------|-------|------|------|
| BR-01 | 号码为123的钥匙 | 顺时针转动 | 开锁   | 上锁   |
| BR-01 | 号码为123的钥匙 | 顺时针转动 | 上锁   | 不变   |
| BR-02 | 号码为123的钥匙 | 逆时针转动 | 开锁   | 不变   |
| BR-02 | 号码为123的钥匙 | 逆时针转动 | 上锁   | 开锁   |

# 负面测试

---

负面测试和 负面测试用例：

展示当输入非预期输入时,产品没有失败(fail)

目的：

产品没有设计、没有预想到的场景，尝试使系统垮掉

# 负面测试.....

---

## 负面测试用例举例

| 序号 | 输入1      | 输入2   | 当前状态 | 预期状态 |
|----|----------|-------|------|------|
| 1  | 某个其它锁的钥匙 | 顺时针转动 | 上锁   | 不变   |
| 2  | 某个其它锁的钥匙 | 逆时针转动 | 开锁   | 不变   |
| 3  | 铁丝       | 逆时针转动 | 开锁   | 不变   |
| 4  | 用石头打击    |       | 上锁   | 不变   |

# 正面测试和负面测试比较

---

|       | 正面测试                | 负面测试               |
|-------|---------------------|--------------------|
| 测试条件  | 根据需求说明规格            | 测试条件都在需求规格说明之外     |
| 目的    | 验证需求规格说明书的需求是否都得到满足 | 通过非法输入尝试搞垮软件       |
| 覆盖率计算 | 覆盖需求和条件             | 没有覆盖率              |
| 挑战性   | 根据规格说明设计测试用例        | 需要高度创造性产生尽可能多的非法输入 |

## 二 黑盒测试用例设计和生成方法

---

- A. 等价类划分法
- B. 因果图法+判定表法
- C. 边界值分析法
- D. 正交试验设计法
- E. 错误推测法
- F. 功能图法
- G. 场景法等
- H. 状态迁移法
- I. 流程分析法
- J. 逐级细分法
- K. 输入域分析法
- L. 输出域分析法
- M. 异常分析
- N. 功能分解法



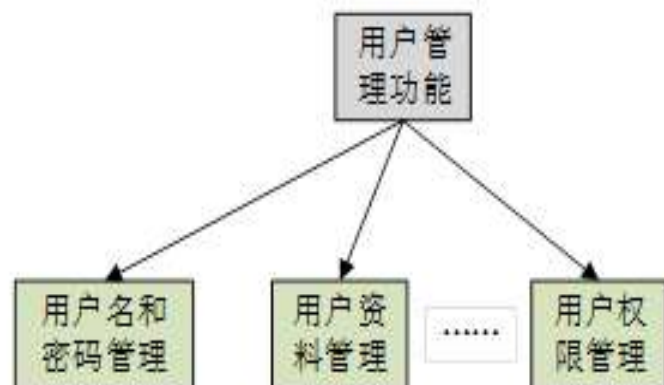
- A. 等价类划分法
- B. 因果图(判定表)法
- C. 边界值分析法

# 功能分解法

功能分解法就是把软件需求中的每一个功能加以分解，分解为功能单位，以功能单位为对象进行测试

用例设计

例子：用户管理系统可以分为如下，当然也可以进一步往下分解。





# A 等价类划分法

---

1. 什么是等价类？
2. 如何进行等价类划分？
3. 如何使用等价类生成测试用例？

# A 等价类划分法...

---

测试面临的挑战：穷举？



等价类划分  
参考规则6

用尽可能少的测试用例发现尽可能多的缺陷。

# A 等价类划分法...

---

## 1. 什么是等价类？

- 等价类划分—equivalence partitioning
- **等价类**：测试相同目标或暴露相同软件缺陷的一组测试。
- **等价类**是指输入域的某个子集，该子集中的每个输入数据对揭露软件中的错误都是等效的，测试等价类的某个代表值就等价于对这一类其他值的测试。也就是说，如果该子集中的某个输入数据能检测出某个错误，那么该子集中的其他输入数据也能检测出同样的错误；反之，如果该子集中的某个输入数据不能检测出错误，那么该子集中的其他输入数据也不能检测出错误。

# A 等价类划分法...

---

## 2. 如何进行等价类划分？

原理：

- 将程序的输入域划分为等价类，以便导出测试用例；
- 它试图通过设计一个测试用例来尽可能发现多个错误，从而减少测试用例数目，降低测试工作量。

等价类（划分）：

- 如果软件行为对一组值来说是相同的，则称这组值为等价类；
- 产生相同预期输出的一组输入值叫一个划分

# A 等价类划分法...

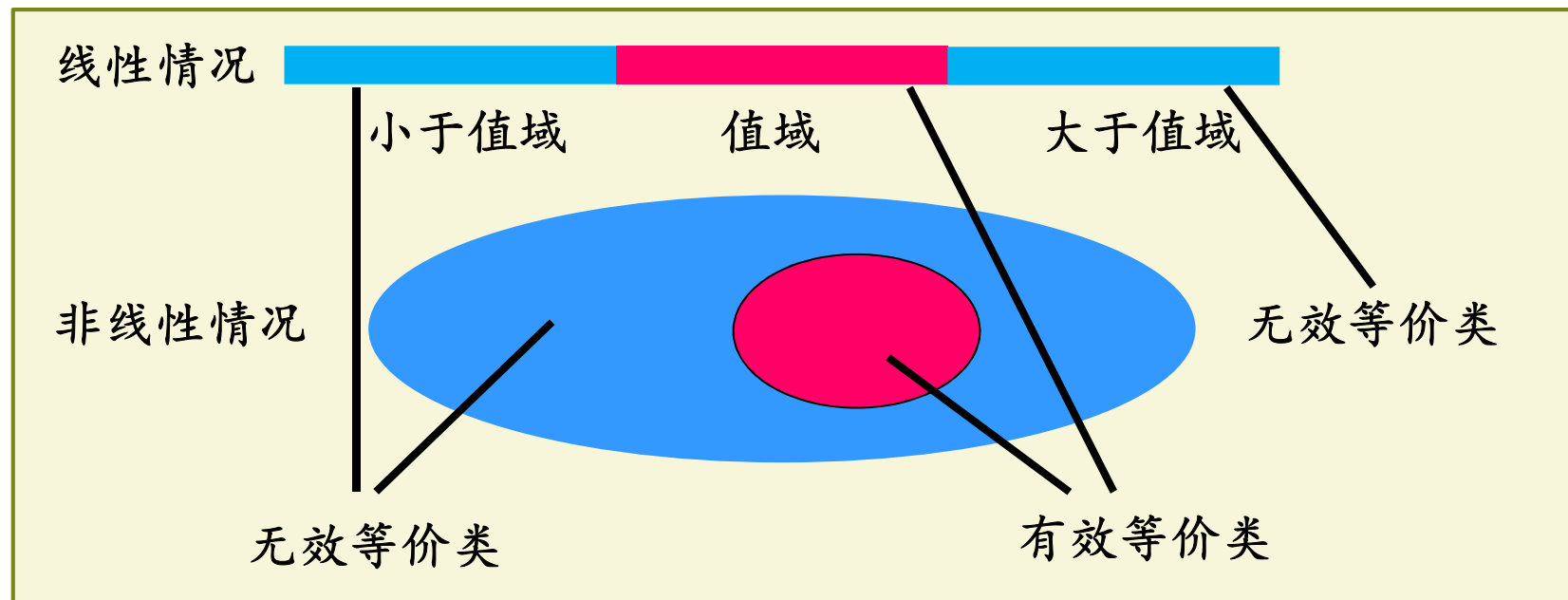
---

## ■ 有效等价类和无效等价类：

- 有效等价类：完全满足产品规格说明的输入数据，即有效的、有意义的输入数据构成的集合。
  - 利用有效等价类可以检验程序是否满足规格说明书。
- 无效等价类：不满足程序输入要求或者无效的输入数据构成的集合。
  - 利用无效等价类可以检验程序是否能够处理非法输入。

# A 等价类划分法...

## ■ 有效等价类和无效等价类



# A 等价类划分法...

## ■ 等价类划分准则

(1) 输入条件是布尔表达式，则可以定义1个有效等价类和1个无效等价类

| 填写产品份额 | 不填写产品份额 |
|--------|---------|
| 有效等价类  | 无效等价类   |

例1. 设置产品信息，其中产品份额必填

例2.  
Office自动  
更正选项



## 二 等价类划分法……

(2) 输入条件代表一个范围，则可以定义1个有效等价类和2个无效等价类

| 权重( $W$ )   |                          |            |
|-------------|--------------------------|------------|
| $W < -1000$ | $-1000 \leq W \leq 1000$ | $W > 1000$ |
| 无效等价类       | 等价有效类                    | 无效等价类      |

例1. 设置风控指标，其中权重设置范围在 $[-1000, 1000]$

例2.  
打印  
范围

打印范围

☒ 所有页面(A)

☐ 当前视图(V)

☐ 当前页面(U)

☐ 页面(G) 1 - 20

子集(B): 范围中的所有页面

☐ 逆页序打印(E)



## A 等价类划分法...

---

(3) 输入数据个数有规定，则可以定义1个有效等价类和2个无效等价类

例1，规定输入3个数构成三角形的3条边

有效等价类：输入边数 = 3

无效等价类：输入边数 < 3 和 输入边数 > 3

# A 等价类划分法...

(4) 输入条件代表集合的某个子集，则可以定义1个有效等价类和1个或多个无效等价类

例1. 邮政编码必须由6位数字构成

有效等价类：6位数字

无效等价类：6位字母    >6位字母    <6位字母  
                  >6位数字    <6位数字    6位字母+数字...

例2. 系统的初始资金只可输入数字

| 等价有效类                                                                                     | 无效等价类  |
|-------------------------------------------------------------------------------------------|--------|
| 输入数字                                                                                      | 输入字母   |
|                                                                                           | 输入汉字   |
|                                                                                           | 输入特殊字符 |
| <a href="http://blog.csdn.net/worixin_36158949">http://blog.csdn.net/worixin_36158949</a> |        |

# A 等价类划分法...

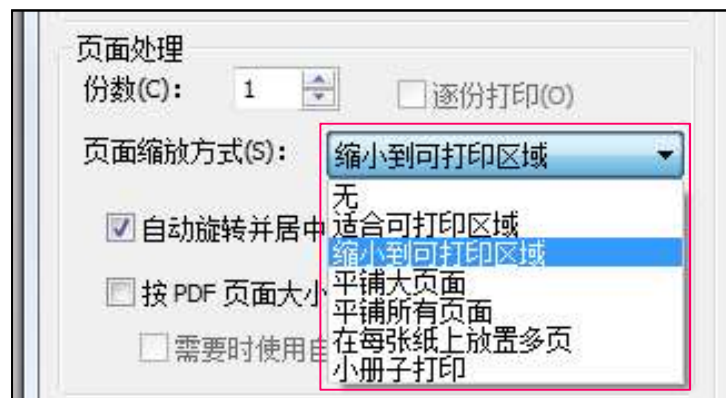
(5) 输入条件代表一组列表形式的数据，则可以定义N个有效等价类和1个无效等价类

例1，中国的直辖市

有效等价类：北京、上海、天津、重庆

无效等价类：非直辖市

例2. 页面处理



# A 等价类划分法...

---

(6) 输入条件代表要求符合某几个规则，则可以定义多个有效等价类和若干个无效等价类

例如，电子邮件地址规则：

必须含有@

@后格式为x.y

地址中不能包含某些符号，如/#&

有效等价类：[字母|数字]1..n@x.y

无效等价类：不含@

@后形式不是x.y

包含逗号

## A 等价类划分法...

| 输入数据类型       | 划分等价类规则                   |                   |
|--------------|---------------------------|-------------------|
| 布尔值          | 1个有效等价类：TRUE              | 1个无效等价类：FALSE     |
| 取值范围         | 1个有效等价类：正确取值范围            | 2个无效等价类：大于和小于取值范围 |
| 数据个数         | 1个有效等价类：正确数据个数            | 2个无效等价类：大于和小于数据个数 |
| 集合           | 1个有效等价类：正确的集合取值           | 1个或多个无效等价类        |
| 需分别处理的一组输入数据 | 多个有效等价类：每个输入数据为1个等价类      | 1个无效等价类           |
| 符合某些规则的输入    | 多个有效等价类：符合某个规则的输入数据为1个等价类 | 若干个无效等价类          |

# A 等价类划分法...

---

## ■ 等价类划分准则

实际情况往往是千变万化的，在遇到具体问题时，可参照上述准则的思想来划分等价类。

# A 等价类划分法...

---

## 3.如何使用等价类生成测试用例？

**Step1:**选择划分准则（范围、取值、布尔、集合...）

**Step2:**根据准则确定有效等价类和无效等价类

**Step3:**从等价类中选取样本数据覆盖所有等价类

**Step4:**根据需求写预期结果

**Step5:**执行测试



每个有效等价类都要覆盖！  
每个无效等价类都要单独覆盖！

# A 等价类划分法...

---

## ■ 等价类划分举例（范围）

例1：人寿保险【参考规则5】

人寿保险费率（基本月保险费0.50）

| 年龄段   | 额外保险费 |
|-------|-------|
| 35岁以下 | 1.65  |
| 35-59 | 2.87  |
| 60岁以上 | 6.00  |



## A 等价类划分法...

---

### ■ 等价类划分举例

人寿保险费率对应等价类

| 序号 | 等价类    | 输入类型 | 测试数据   | 预期结果      |
|----|--------|------|--------|-----------|
| 1  | 低于35岁  | 有效   | 26, 12 | 月保险费=2.15 |
| 2  | 35-59岁 | 有效   | 37     | 月保险费=3.37 |
| 3  | 大于60岁  | 有效   | 65, 90 | 月保险费=6.50 |
| 4  | 负年岁    | 无效   | -23    | 警告信息      |

# A 等价类划分法...

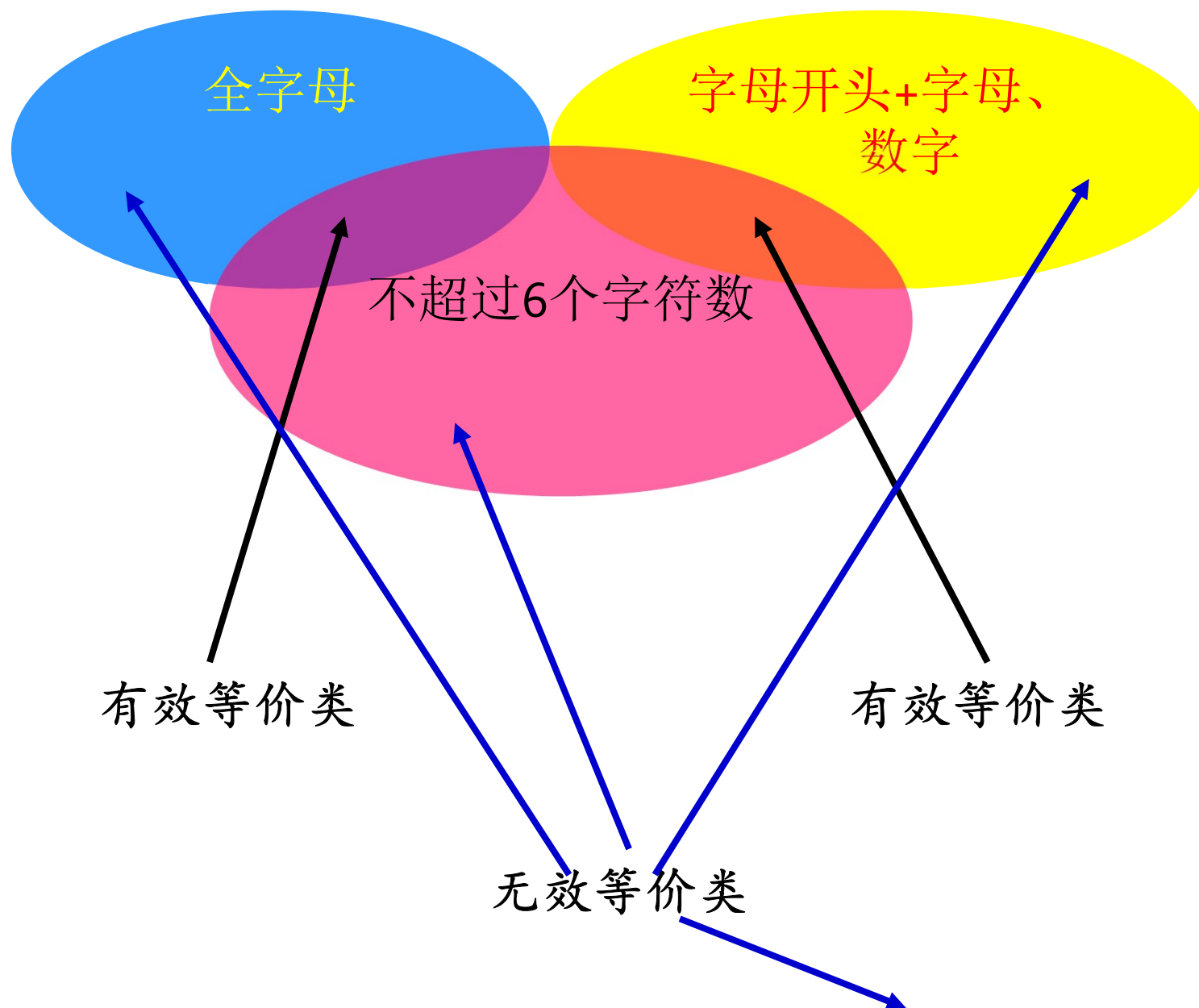
---

## ■ 等价类划分举例

例2：注册用户名【参考规则6】

合法的用户名要求：

- (1) 由字母开头
- (2) 后跟字母或数字的任意组合构成
- (3) 有效字符数不超过6个。



# A 等价类划分法...

---

2个有效等价类:

(1) 用户名: {0<全字母<=6}

John, Jerry, Kenedy

(2) 用户名: {0<字母开头+数字<=6}

u001, user01

# A 等价类划分法...

---

4个无效等价类:

(1) 字母串, 且长度超过6

userabcd

(2) 字母开头的字符、数字组合串, 且长度超过6

user01587

(3) 长度不超过6的数字开头串

010ah

(4) 其他

长度不超过6的数字开头的字符串集合、空字符串、非法字符串

# A 等价类划分法...

---

## ■ 思考题:

等价类划分考虑输入条件还是输出条件?

考虑输出条件有助于对输入域进行精细的等价类划分。

# 经典例子：判断三角形类型

- 程序：输入三角形三边的长度，判定三角形的类型为一般三角形、等腰三角形、等边三角形或非三角形。

```
compute(int a, int b, int c ){  
    //a,b,c >0;    a+b>c;  
  
    ...  
}
```

【参考规则6】

2025/4/8

Python

```
1 def classify_triangle(a, b, c):  
2     """判断三角形类型"""  
3     # 将三边排序以简化判断逻辑  
4     sides = sorted([a, b, c])  
5  
6     # 检查是否为三角形  
7     if sides[0] + sides[1] <= sides[2]:  
8         return "非三角形"  
9  
10    # 判断等边三角形  
11    if a == b == c:  
12        return "等边三角形"  
13  
14    # 判断等腰三角形  
15    if a == b or b == c or a == c:  
16        return "等腰三角形"  
17  
18    # 其余情况为一般三角形  
19    return "一般三角形"  
20  
21 def get_valid_input():  
22     """获取有效的三边输入"""  
23     while True:  
24         input_str = input("请输入三角形的三边长度 (用空格分隔): ")  
25         parts = input_str.split()  
26  
27         # 检查输入数量  
28         if len(parts) != 3:  
29             print("错误: 必须输入三个数字")  
30             continue  
31  
32         try:  
33             # 转换为浮点数并检查正数  
34             sides = list(map(float, parts))  
35             if any(side <= 0 for side in sides):  
36                 print("错误: 边长必须大于0")  
37                 continue  
38             return sides  
39         except ValueError:  
40             print("错误: 包含无效的非数字输入")  
41  
42 if __name__ == "__main__":  
43     # 获取输入并判断类型  
44     a, b, c = get_valid_input()  
45     result = classify_triangle(a, b, c)  
46     print(f"三角形类型: {result}")
```

| 输入三个整数 | 有效等价类型 | 号码 | 无效等价类                                                                                                                                                                                                                      | 号码 |
|--------|--------|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
|        | 整数     | 1  | 一边为非整数 { <ul style="list-style-type: none"> <li>a 为非整数</li> <li>b 为非整数</li> <li>c 为非整数</li> </ul> 两边为非整数 { <ul style="list-style-type: none"> <li>a,b 为非整数</li> <li>b,c 为非整数</li> <li>a,c 为非整数</li> </ul> 三边 a, b, c 均为非整数 | 12 |
|        |        |    |                                                                                                                                                                                                                            | 13 |
|        |        |    |                                                                                                                                                                                                                            | 14 |
|        |        |    |                                                                                                                                                                                                                            | 15 |
|        |        |    |                                                                                                                                                                                                                            | 16 |
|        |        |    |                                                                                                                                                                                                                            | 17 |
|        |        |    |                                                                                                                                                                                                                            | 18 |
|        | 三个数    | 2  | 只给一边 { <ul style="list-style-type: none"> <li>只给 a</li> <li>只给 b</li> <li>只给 c</li> </ul> 只给两边 { <ul style="list-style-type: none"> <li>只给 ab</li> <li>只给 bc</li> <li>只给 ac</li> </ul> 给出三个以上                              | 19 |
|        |        |    |                                                                                                                                                                                                                            | 20 |
|        |        |    |                                                                                                                                                                                                                            | 21 |
|        |        |    |                                                                                                                                                                                                                            | 22 |
|        |        |    |                                                                                                                                                                                                                            | 23 |
|        |        |    |                                                                                                                                                                                                                            | 24 |
|        |        |    |                                                                                                                                                                                                                            | 25 |
|        | 非零数    | 3  | 一边为零 { <ul style="list-style-type: none"> <li>a 为 0</li> <li>b 为 0</li> <li>c 为 0</li> </ul> 二边为零 { <ul style="list-style-type: none"> <li>a,b 为 0</li> <li>b,c 为 0</li> <li>a,c 为 0</li> </ul> 三边 a,b,c 均为 0              | 26 |
|        |        |    |                                                                                                                                                                                                                            | 27 |
|        |        |    |                                                                                                                                                                                                                            | 28 |
|        |        |    |                                                                                                                                                                                                                            | 29 |
|        |        |    |                                                                                                                                                                                                                            | 30 |
|        |        |    |                                                                                                                                                                                                                            | 31 |
|        |        |    |                                                                                                                                                                                                                            | 32 |



|         |                                                                           |    |                                                                                                                                        |    |
|---------|---------------------------------------------------------------------------|----|----------------------------------------------------------------------------------------------------------------------------------------|----|
|         | 正数                                                                        | 4  | 一边 $<0$ $\begin{cases} a<0 \\ b<0 \\ c<0 \end{cases}$                                                                                  | 33 |
|         |                                                                           |    | 二边 $<0$ $\begin{cases} a<0 \text{ 且 } b<0 \\ a<0 \text{ 且 } c<0 \\ b<0 \text{ 且 } c<0 \end{cases}$                                     | 34 |
|         |                                                                           |    |                                                                                                                                        | 35 |
|         |                                                                           |    |                                                                                                                                        | 36 |
|         |                                                                           |    |                                                                                                                                        | 37 |
|         |                                                                           |    |                                                                                                                                        | 38 |
|         |                                                                           |    | 三边均 $<0$ : $a<0$ 且 $b<0$ 且 $c<0$                                                                                                       | 39 |
| 构成一般三角形 | $a+b>c$                                                                   | 5  | $\begin{cases} a+b<c \\ a+b=c \end{cases}$<br>$\begin{cases} b+c<a \\ b+c=a \end{cases}$<br>$\begin{cases} a+c<b \\ a+c=b \end{cases}$ | 40 |
|         | $b+c>a$                                                                   | 6  |                                                                                                                                        | 41 |
|         | $a+c>b$                                                                   | 7  |                                                                                                                                        | 42 |
|         |                                                                           |    |                                                                                                                                        | 43 |
|         |                                                                           |    |                                                                                                                                        | 44 |
|         |                                                                           |    |                                                                                                                                        | 45 |
| 构成等腰三角形 | $a=b$ $\left. \begin{matrix} b=c \\ a=c \end{matrix} \right\}$ 且两边之和大于第三边 | 8  |                                                                                                                                        |    |
|         |                                                                           | 9  |                                                                                                                                        |    |
|         |                                                                           | 10 |                                                                                                                                        |    |
| 构成等腰三角形 | $a=b=c$                                                                   | 11 |                                                                                                                                        |    |

# 有效等价类

每个有效等价类都要覆盖！  
每个无效等价类都要单独覆盖！

- 覆盖有效等价类的测试用例：

| <b>a</b> | <b>b</b> | <b>c</b> | 覆盖等价类号码           |
|----------|----------|----------|-------------------|
| 3        | 4        | 5        | (1) -- (7)        |
| 4        | 4        | 5        | (1) -- (7) , (8)  |
| 4        | 5        | 5        | (1) -- (7) , (9)  |
| 5        | 4        | 5        | (1) -- (7) , (10) |
| 4        | 4        | 4        | (1) -- (7) , (11) |

# 无效等价类

每个有效等价类都要覆盖！  
每个无效等价类都要单独覆盖！

| a   | b   | c   | 覆盖等价类号码 | a  | b  | c  | 覆盖等价类号码 |
|-----|-----|-----|---------|----|----|----|---------|
| 2.5 | 4   | 5   | 12      | 0  | 0  | 5  | 29      |
| 3   | 4.5 | 5   | 13      | 3  | 0  | 0  | 30      |
| 3   | 4   | 5.5 | 14      | 0  | 4  | 0  | 31      |
| 3.5 | 4.5 | 5   | 15      | 0  | 0  | 0  | 32      |
| 3   | 4.5 | 5.5 | 16      | -3 | 4  | 5  | 33      |
| 3.5 | 4   | 5.5 | 17      | 3  | -4 | 5  | 34      |
| 4.5 | 4.5 | 5.5 | 18      | 3  | 4  | -5 | 35      |
| 3   |     |     | 19      | -3 | -4 | 5  | 36      |
|     | 4   |     | 20      | -3 | 4  | -5 | 37      |
|     |     | 5   | 21      | 3  | -4 | -5 | 38      |
| 3   | 4   |     | 22      | -3 | -4 | -5 | 39      |
|     | 4   | 5   | 23      | 3  | 1  | 5  | 40      |
| 3   |     | 5   | 24      | 3  | 2  | 5  | 41      |
| 3   | 4   | 5   | 25      | 3  | 1  | 1  | 42      |
| 0   | 4   | 5   | 26      | 3  | 2  | 1  | 43      |
| 3   | 0   | 5   | 27      | 1  | 4  | 2  | 44      |
| 3   | 4   | 0   | 28      | 3  | 4  | 1  | 45      |

## B 因果图(判定表)法

---

1. 什么是因果图？
2. 如何根据软件需求画出因果图？
3. 如何生成判定表以及测试用例？

## B 因果图(判定表)...

---

### ■ 因果分析法背景

测试挑战：多个输入条件的关联问题

例1：火车票价查询：终点、车次、硬座/卧铺、普通票/学生票等多个因素决定

例2：包裹费用计算：快递/EMS、重量、距离等

组合测试面临处理大量无效测试用例的现实

## B 因果图(判定表)...

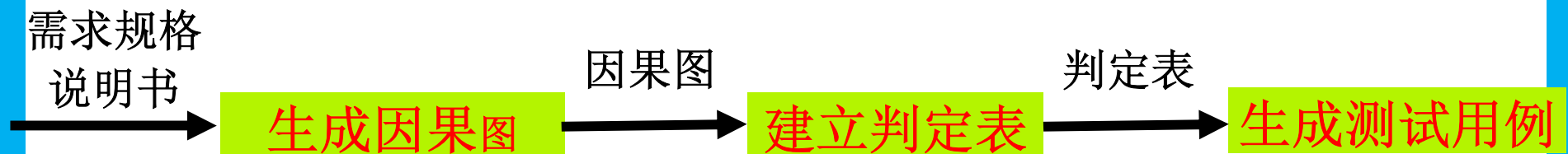
---

### ■ 因果分析法基础

原理:

- (1) 软件的输入和输出之间存在因果逻辑关系，可以用因果图（**cause-effect diagram**）刻画；
- (2) 因果图可从规格说明书中获得

过程:



## B 因果图(判定表)...

---

### ■ 因果图的表示

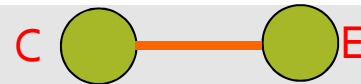
符号： $C_i$ 表示原因， $E_i$ 表示结果，原因和结果的4种关系：

(1) 恒等（—）：

若 $C_i$ 出现，则 $E_i$ 出现；

若 $C_i$ 不出现，则 $E_i$ 也不出现

恒等（—）



## B 因果图(判定表)...

---

(2) 非 ( $\sim$ ) :

若C出现, 则E不出现;  
若C不出现, 则E出现

非 ( $\sim$ )



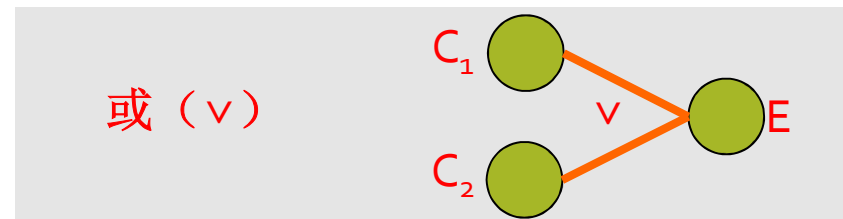


## B 因果图(判定表)...

---

(3) 或 ( $\vee$ ) :

若几个 $C_i$ 中有一个出现, 则E出现;  
若几个 $C_i$ 都不出现, 则E不出现

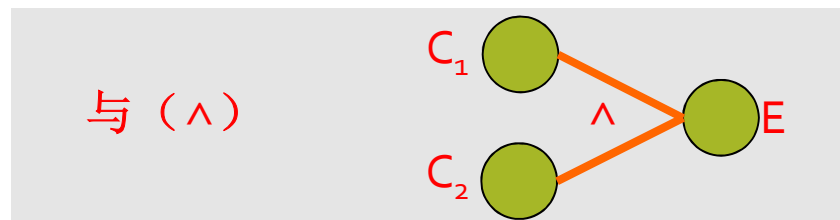


## B 因果图(判定表)...

---

(4) 与 ( $\wedge$ ) :

若几个 $C_i$ 都出现, 则 $E$ 出现;  
若其中一个 $C_i$ 不出现, 则 $E$ 不出现

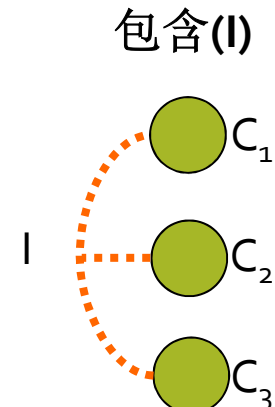
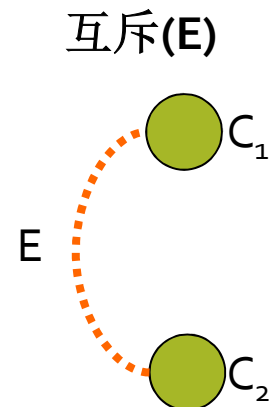


## B 因果图(判定表)...

---

另外，因果图有4种输入约束：

- (1) 互斥(E):多个原因不能同时成立，最多有一个能成立；即 $C_i$ 不能同时为1
- (2) 包含(I):多个原因中至少有一个必须成立；即 $C_i$ 不能同时为0

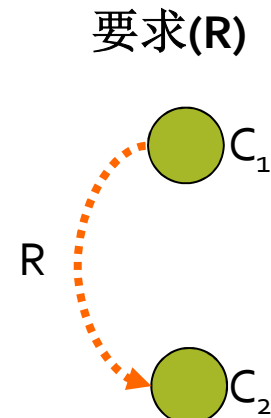
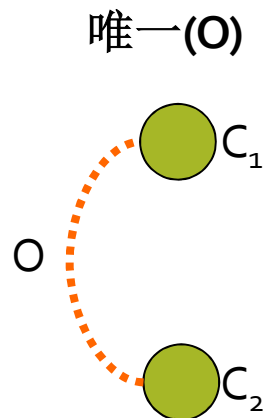


## B 因果图(判定表)...

---

(3) 唯一(O):多个原因中必须有一个且只有一个成立; 即 $C_i$ 只有一个为1

(4) 要求(R):当 $C_1$ 成立,  $C_2$ 也必须成立

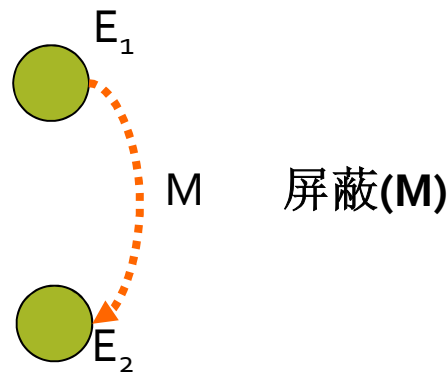


## B 因果图(判定表)...

---

还有1种输出约束:

(1) 屏蔽(M):当 $E_1$ 是1时,  $E_2$ 必须是0; 当 $E_1$ 是0,  $E_2$ 的值不定



## B 因果图(判定表)...

---

### ■因果图分析练习1

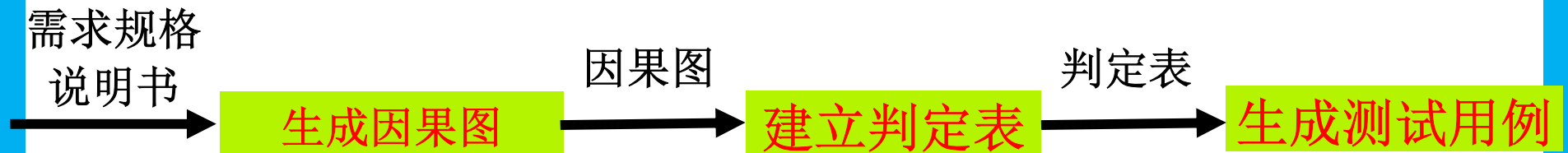
#### 文件管理系统规格说明

- (1) 文件第一列的字符必须是一个A或B，且文件第二列的字符必须是一个数字；
- (2) 若符合上述情况，则打印文件已被修改的消息；
- (3) 若第一个字符不正确，则打印X12消息；
- (4) 若第二个不是数字，则打印X13消息；

## B 因果图(判定表)...

---

过程:



## B 因果图(判定表)...

---

### ■ 原因和结果

原因:

C1:第1列的字符是A

C2:第1列的字符是B

C3:第2列的字符是数字

结果:

E1:文件修改过

E2:打印消息X12

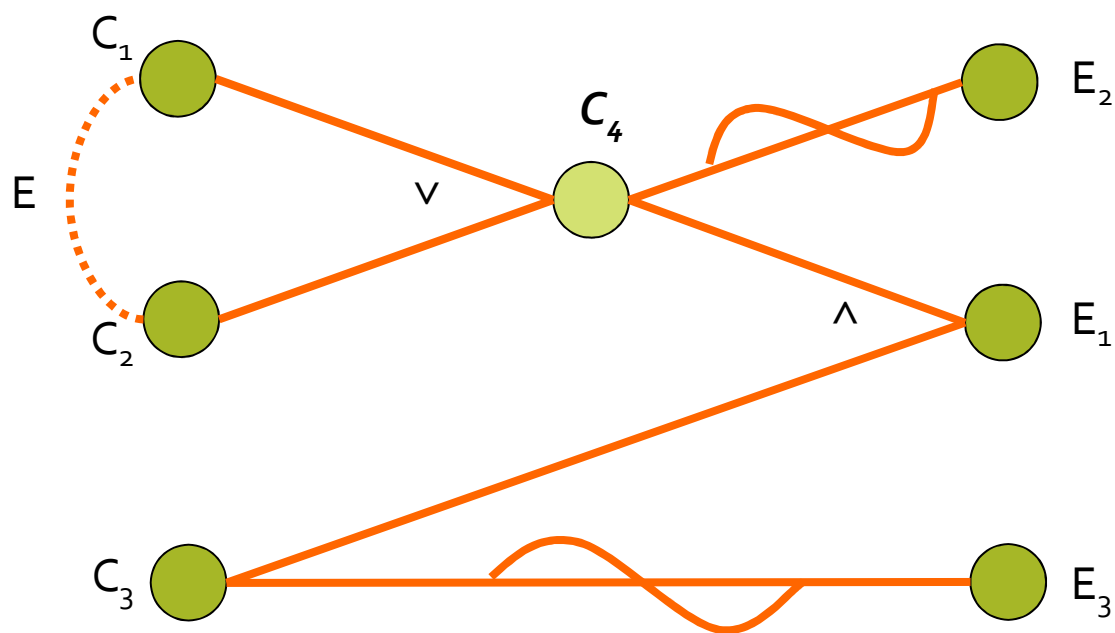
E3:打印消息X13



## B 因果图(判定表)...

---

### ■ 生成因果图



## B 因果图(判定表)...

### ■ 因果图列表

因果图列表：原因的所有组合及相应的结果

注意：某些原因的组合可能不存在

| 因果图列表示例      |    |   |   |   |   |   |   |   |   |
|--------------|----|---|---|---|---|---|---|---|---|
|              |    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 原因<br>(可能输入) | C1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
|              | C2 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
|              | C3 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 结果<br>(预期输出) | E1 | 0 | 0 | 0 | 1 | 0 | 1 |   |   |
|              | E2 | 1 | 1 | 0 | 0 | 0 | 0 |   |   |
|              | E3 | 1 | 0 | 1 | 0 | 1 | 0 |   |   |

## B 因果图(判定表)...

### ■生成判定表

方法： 原因：判定表中的条件； 结果：判定表中的行动； 判定规则：原因与结果的组合。

| 判定表示例        |    |   |   |   |   |   |   |
|--------------|----|---|---|---|---|---|---|
|              |    | 1 | 2 | 3 | 4 | 5 | 6 |
| 条件<br>(有效输入) | C1 | 0 | 0 | 0 | 0 | 1 | 1 |
|              | C2 | 0 | 0 | 1 | 1 | 0 | 0 |
|              | C3 | 0 | 1 | 0 | 1 | 0 | 1 |
| 行动<br>(预期输出) | A1 | 0 | 0 | 0 | 1 | 0 | 1 |
|              | A2 | 1 | 1 | 0 | 0 | 0 | 0 |
|              | A3 | 1 | 0 | 1 | 0 | 1 | 0 |

## B 因果图(判定表)...

---

### ■生成测试用例

- (1)判定表中的条件：测试用例的输入条件
- (2)判定表中的行动：测试用例的期望输出
- (3)一条判定规则：一个测试用例

## B 因果图(判定表)...

|              |      | 1  | 2  | 3  | 4  | 5  | 6  |
|--------------|------|----|----|----|----|----|----|
| 条件<br>(合法输入) | C1   | 0  | 0  | 0  | 0  | 1  | 1  |
|              | C2   | 0  | 0  | 1  | 1  | 0  | 0  |
|              | C3   | 0  | 1  | 0  | 1  | 0  | 1  |
| 行动<br>(预期输出) | A1   | 0  | 0  | 0  | 1  | 0  | 1  |
|              | A2   | 1  | 1  | 0  | 0  | 0  | 0  |
|              | A3   | 1  | 0  | 1  | 0  | 1  | 0  |
| 测试用例         | 测试输入 | DY | C2 | BN | B5 | AM | A3 |
|              | 测试输出 |    |    |    |    |    |    |

## B 因果图(判定表)...

---

### ■ 因果图分析练习2

#### 饮料自动售货机规格说明

- (1) 若投入5角钱或1元钱的硬币，压下〔橙汁〕或〔啤酒〕的按钮，则相应的饮料就送出来。
- (2) 若售货机没有零钱找，则一个显示〔零钱找完〕的红灯亮，这时在投入1元硬币并压下按钮后，饮料不送出来而且1元硬币也退出来；
- (3) 若有零钱找，则〔零钱找完〕的红灯灭，在送出饮料的同时退还5角硬币。

## B 因果图(判定表)...

---

### ■生成因果图

#### 原因:

C1:售货机有零钱找

C2:投入1元硬币

C3:投入5角硬币

C4:压下橙汁按钮

C5:压下啤酒按钮

#### 结果:

E1:售货机〔零钱找完〕灯亮

E2:退还1元硬币

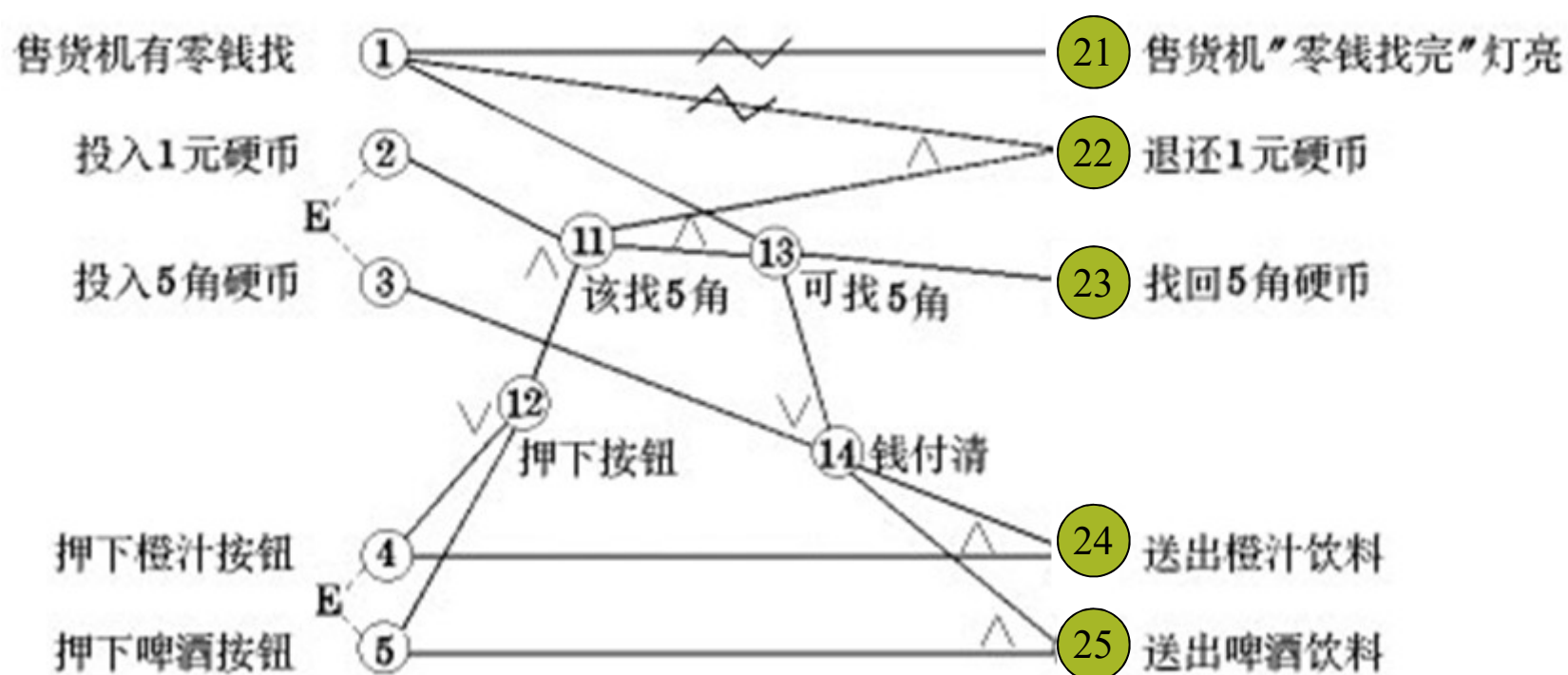
E3:退还5角硬币

E4:送出橙汁饮料

E5:送出啤酒饮料

## B 因果图(判定表)...

### ■生成因果图





## B 因果图(判定表)...

### ■ 生成判定表和测试用例

C1:售货机有零钱找

C2:投入1元硬币

C3:投入5角硬币

C4:压下橙汁按钮

C5:压下啤酒按钮

| 序号               |   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 20 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 30 | 1 | 2 |   |
|------------------|---|---|---|---|---|---|---|---|---|---|----|---|---|---|---|---|---|---|---|---|----|---|---|---|---|---|---|---|---|---|----|---|---|---|
| 条<br>件           | ① | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0 | 0 |   |
|                  | ② | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0  | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1  | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0  | 0 | 0 |   |
|                  | ③ | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1  | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1  | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0  | 0 | 0 |   |
|                  | ④ | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1  | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0  | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1  | 1 | 0 | 0 |
|                  | ⑤ | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0  | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0  | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0  | 1 | 0 | 1 |
| 中<br>间<br>结<br>果 | ⑪ |   |   |   |   | 1 | 1 | 0 |   |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 1 | 1 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
|                  | ⑫ |   |   |   |   | 1 | 1 | 0 |   |   | 1  | 1 | 0 |   | 1 | 1 | 0 |   |   |   |    |   | 1 | 1 | 0 |   | 1 | 1 | 0 |   | 1  | 1 | 0 |   |
|                  | ⑬ |   |   |   |   | 1 | 1 | 0 |   |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
|                  | ⑭ |   |   |   |   | 1 | 1 | 0 |   |   | 1  | 1 | 1 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 1 | 1 | 1 |   | 0  | 0 | 0 |   |
| 结<br>果           | ⑳ |   |   |   |   | 0 | 0 | 0 |   |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 1 | 1 | 1 |   | 1 | 1 | 1 |   | 1  | 1 | 1 |   |
|                  | ㉑ |   |   |   |   | 0 | 0 | 0 |   |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 1 | 1 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
|                  | ㉒ |   |   |   |   | 1 | 1 | 0 |   |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
|                  | ㉓ |   |   |   |   | 1 | 0 | 0 |   |   | 1  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 1 | 0 | 0 |   | 0  | 0 | 0 |   |
|                  | ㉔ |   |   |   |   | 0 | 1 | 0 |   |   | 0  | 1 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 0 | 1 | 0 |   | 0  | 0 | 0 |   |
| 测试<br>用例         |   |   |   |   |   | Y | Y | Y |   |   | Y  | Y | Y |   | Y | Y |   |   |   |   |    |   | Y | Y | Y |   | Y | Y | Y |   | Y  | Y |   |   |

阴影部分表示因违反约束条件的不可能出现的情况，删去。

第16列与第32列因什么动作也没做，也删去（条件没发生）。

最后可根据剩下的16列作为确定测试用例的依据

## B 因果图(判定表)...

---

### ■ 因果分析法总结

**Step1:** 分析规格说明书, 识别原因和结果

**Step2:** 在因果图连接原因和结果

**Step3:** 标明原因之间以及结果之间的约束条件

**Step4:** 因果图转换为因果图列表进而生成判定表

**Step5:** 判定表的规则转换为测试用例

## B 因果图(判定表)...

---

### ■判定表组成

4种成份：

- (1) 条件桩：列出所有可能问题（条件）
- (2) 条件项：列出条件所有可能取值
- (3) 动作桩：列出可能采取的操作
- (4) 动作项：指出在条件项的各种取值情况下应采取的动作

判定规则：贯穿条件项和动作项的一系列

## B 因果图(判定表)...

## 决策规则

| 序号   |      | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 20 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 30 | 1 | 2 |   |
|------|------|---|---|---|---|---|---|---|---|---|----|---|---|---|---|---|---|---|---|---|----|---|---|---|---|---|---|---|---|---|----|---|---|---|
| 条件桩  | 条    | ① | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0 | 0 |   |
|      | 件    | ② | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1  | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0  | 0 | 0 |   |
|      |      | ③ | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1  | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1  | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0  | 0 | 0 | 0 |
|      |      | ④ | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1  | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0  | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1  | 1 | 0 | 0 |
|      |      | ⑤ | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1  | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1  | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1  | 0 | 1 | 0 |
| 动作桩  | 中间结果 | ⑪ |   |   |   |   | 1 | 1 | 0 |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 1 | 1 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
|      | ⑫    |   |   |   |   |   | 1 | 1 | 0 |   | 1  | 1 | 0 |   | 1 | 1 | 0 |   |   |   |    |   | 1 | 1 | 0 |   | 1 | 1 | 0 |   | 1  | 1 | 0 |   |
|      | ⑬    |   |   |   |   |   | 1 | 1 | 0 |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
|      | ⑭    |   |   |   |   |   | 1 | 1 | 0 |   | 1  | 1 | 1 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 1 | 1 | 1 |   | 0  | 0 | 0 |   |
|      | 结果   | ⑲ |   |   |   |   |   | 0 | 0 | 0 |    | 0 | 0 | 0 |   | 0 | 0 | 0 |   |   |    |   |   | 1 | 1 | 1 |   | 1 | 1 | 1 |    | 1 | 1 | 1 |
| ⑳    |      |   |   |   |   |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 1 | 1 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
| ㉑    |      |   |   |   |   |   | 1 | 1 | 0 |   | 0  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 0 | 0 | 0 |   | 0  | 0 | 0 |   |
| ㉒    |      |   |   |   |   |   | 1 | 0 | 0 |   | 1  | 0 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 1 | 0 | 0 |   | 0  | 0 | 0 |   |
| ㉓    |      |   |   |   |   |   | 0 | 1 | 0 |   | 0  | 1 | 0 |   | 0 | 0 | 0 |   |   |   |    |   | 0 | 0 | 0 |   | 0 | 1 | 0 |   | 0  | 0 | 0 |   |
| 测试用例 |      |   |   |   |   |   | Y | Y | Y |   |    | Y | Y | Y |   | Y | Y |   |   |   |    |   | Y | Y | Y |   | Y | Y | Y |   | Y  | Y |   |   |

## B 因果图(判定表)...

---

### ■判定表简化

简化目标：合并相似规则

相似规则判断：

有两条或以上规则具有相同动作，并且在条件项之间存在极大相似，便可以合并

“—”：表示合并后该条件项与取值无关，称“无关条件”

## B 因果图(判定表)...

---

■ 例如：

对功率大于100马力的机器、维修记录不全或已运行10年以上的机器，应给予优先的维修处理。

条件1： 机器功率大小

条件2： 维修记录

条件3： 运行时间

动作1： 优先维修

动作2： 正常维修

## B 因果图(判定表)...

### ■判定表构造

初始化判定表

|    |           | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|----|-----------|---|---|---|---|---|---|---|---|
| 条件 | 功率大于100马力 | Y | Y | Y | Y | N | N | N | N |
|    | 维修记录不全    | Y | Y | N | N | Y | Y | N | N |
|    | 运行时间超过10年 | Y | N | Y | N | Y | N | Y | N |
| 动作 | 优先维修      | √ | √ | √ | √ | √ | √ | √ |   |
|    | 正常维修      |   |   |   |   |   |   |   | √ |

## B 因果图(判定表)...

### ■判定表化简

| 初始化判定表化简 |           |         |     |   |   |
|----------|-----------|---------|-----|---|---|
|          |           | 1 2 3 4 | 5 6 | 7 | 8 |
| 条件       | 功率大于100马力 | Y       | N   | N | N |
|          | 维修记录不全    | --      | Y   | N | N |
|          | 运行时间超过10年 | --      | --  | Y | N |
| 动作       | 优先维修      | √       | √   | √ |   |
|          | 正常维修      |         |     |   | √ |



## B 因果图(判定表)...

---

### ■ 因果图分析练习2

移动公司为促进业务发展发行了各种优惠卡，其中包括钻石卡、金卡、银卡3种。发卡规则为：

从未发生过话费拖欠，且每月通话费在500元（含）以上者可获钻石卡，每月通话费在200元（含）以上者可获金卡，每月通话费在100元（含）以上者可获银卡；

发生过话费拖欠的不能获钻石卡，若能在规定时间内缴清欠款，且每月通话费在300元（含）以上者可获金卡，每月通话费在200元（含）以上者可获银卡；若发生话费拖欠且并未能在规定时间内缴清欠款，无论每月话费多少都不能获得优惠卡。

## B 因果图(判定表)...

|           |                 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|-----------|-----------------|---|---|---|---|---|---|---|---|---|----|----|
| 输入/<br>条件 | 是否曾话费拖欠         | Y | Y | Y | Y | Y | Y | N | N | N | N  | N  |
|           | 是否按时结欠          | N | Y | Y | Y | Y | Y | — | — | — | —  | —  |
|           | 通话费用 $\geq 500$ | — | Y | N | N | N | N | Y | N | N | N  | N  |
|           | 通话费用[300, 500)  | — | N | Y | N | N | N | N | Y | N | N  | N  |
|           | 通话费用[200, 300)  | — | N | N | Y | N | N | N | N | Y | N  | N  |
|           | 通话费用[100, 200)  | — | N | N | N | Y | N | N | N | N | Y  | N  |
|           | 通话费用 $< 100$    | — | N | N | N | N | Y | N | N | N | N  | Y  |
| 输出/<br>动作 | 钻石卡             |   |   |   |   |   |   | √ |   |   |    |    |
|           | 金卡              |   | √ | √ |   |   |   |   | √ | √ |    |    |
|           | 银卡              |   |   |   | √ |   |   |   |   |   | √  |    |
|           | 无卡              | √ |   |   |   | √ | √ |   |   |   |    | √  |

## B 因果图(判定表)...

|           |                   | 1 | 2, 3 | 4 | 5, 6 | 7 | 8, 9 | 10 | 11 |
|-----------|-------------------|---|------|---|------|---|------|----|----|
| 输入/<br>条件 | 是否曾话费拖欠           | Y | Y    | Y | Y    | N | N    | N  | N  |
|           | 是否按时结欠            | N | Y    | Y | Y    | — | —    | —  | —  |
|           | 通话费用 $\geq 500$   | — | —    | N | N    | Y | N    | N  | N  |
|           | 通话费用 $\geq 300$   | — | Y    | N | N    | N | —    | N  | N  |
|           | 通话费用 $\geq 200$   | — | N    | Y | N    | N | Y    | N  | N  |
|           | 通话费用 $[100, 200)$ | — | N    | N | —    | N | N    | Y  | N  |
|           | 通话费用 $< 100$      | — | N    | N | —    | N | N    | N  | Y  |
| 输出/<br>动作 | 钻石卡               |   |      |   |      | √ |      |    |    |
|           | 金卡                |   | √    |   |      |   | √    |    |    |
|           | 银卡                |   |      | √ |      |   |      | √  |    |
|           | 无卡                | √ |      |   | √    |   |      |    | √  |

## B 因果图(判定表)...

|           |                | 1  | 2, 3      | 4         | 5, 6      | 7        | 8, 9     | 10       | 11      |
|-----------|----------------|----|-----------|-----------|-----------|----------|----------|----------|---------|
| 输入<br>/条件 | 是否曾话费拖欠        | Y  | Y         | Y         | Y         | N        | N        | N        | N       |
|           | 是否按时结欠         | N  | Y         | Y         | Y         | —        | —        | —        | —       |
|           | 通话费用≥500       | —  | —         | N         | N         | Y        | N        | N        | N       |
|           | 通话费用≥300       | —  | Y         | N         | N         | N        | —        | N        | N       |
|           | 通话费用≥200       | —  | N         | Y         | N         | N        | Y        | N        | N       |
|           | 通话费用[100, 200) | —  | N         | N         | —         | N        | N        | Y        | N       |
|           | 通话费用<100       | —  | N         | N         | —         | N        | N        | N        | Y       |
| 输出<br>/动作 | 钻石卡            |    |           |           |           | √        |          |          |         |
|           | 金卡             |    | √         |           |           |          | √        |          |         |
|           | 银卡             |    |           | √         |           |          |          | √        |         |
|           | 无卡             | √  |           |           | √         |          |          |          | √       |
| 用例        |                | 欠费 | YY30<br>0 | YY<br>200 | YY<br>100 | N<br>500 | N<br>200 | N<br>100 | N<br>90 |
| 结果        |                | 无卡 | 金卡        | 银卡        | 无卡        | 钻卡       | 金卡       | 银卡       | 无卡      |

## C 边界值分析法

---

1. 什么是边界？
2. 什么是边界值分析？
3. 为什么需要进行边界值分析？
4. 如何使用边界值生成测试用例？
5. 边界值附近测试用例（数据）确认方法

# 1.什么是边界?

---

- 例如
  - 对16-bit 的整数而言 32767 和 -32768 是边界;
  - 屏幕上光标在最左上、最右下位置;
  - 报表的第一行和最后一行;
  - 数组元素的第一个和最后一个;
  - 循环的第 0 次、第 1 次和倒数第 2 次、最后一次。

## 2.什么是边界值分析法?

---

- 边界值分析法就是对输入或输出的边界值进行测试的一种黑盒测试方法。通常边界值分析法是作为对等价类划分法的补充，这种情况下，其测试用例来自等价类的边界。

### 3. 为什么需要进行边界值分析?

---

- 软件的两个主要缺陷源：1) 条件；2) 边界

- 条件：变量取值需要满足的约束
- 边界：变量的各种“极限取值”

- 边界值分析：

- 能有效捕获出现在边界处缺陷的一种测试方法；利用并扩展了缺陷更容易出现在边界处的理念



缺陷容易出现在边界处的原因：

- (1) 使用比较操作符时未仔细分析
- (2) 多种循环和条件检查方法引起的困惑
- (3) 对边界附近需求的理解不够

.....

## 4. 如何使用边界值设计测试用例？

---

- 使用边界值分析方法设计测试用例，**首先**应确定边界情况。例如，通常输入和输出**等价类的边界**，就是应重点测试的边界情况。**然后**应当选取正好等于，刚刚大于或刚刚小于边界的值作为测试数据，而不是选取等价类中的典型值或任意值作为测试数据。

## ■ 例如

- ◆ 第1个减1/最后1个加1
- ◆ 开始减1/完成加1
- ◆ 最小值减1/最大值加1

## ■ 测试边界：

- ◆ 测试临近边界的有效数据，测试最后一个可能有效的数据，同时测试刚超过边界的无效数据

## ■ 边界条件类型

### ◆ 数据类型:

- 数值 速度 字符 地点 位置 尺寸 数量

### ◆ 特征:

- 第一个/最后一个 最小值/最大值
- 开始/完成 超过/在内
- 空/满 最短/最长 最慢/最快 最早/最迟
- 最大/最小

## 一些常见的边界值

|                                                           |
|-----------------------------------------------------------|
| 报表的第一行、最后一行 或 第一列、最后一列                                    |
| 循环的开始、结束                                                  |
| 屏幕光标/鼠标的左上角、右下角                                           |
| <b>Int, long, unsigned int</b> 整数的最小，最大值                  |
| <b>e.g. [signed] long int</b> 4位   -2147483648~2147483647 |
| <b>unsigned long [int]</b> 4位        0~4294967295         |

- 测试用例（数据）确认

边界值附近数据的确认举例

|       | 边界值附近数据                              | 测试用例设计思路                                                  |
|-------|--------------------------------------|-----------------------------------------------------------|
| 字符    | Min-1, Min, Min+1<br>Max-1,Max,Max+1 | 若文本输入框允许输入1-255个字符。<br><br>则边界值测试用例：<br>0,1,2,254,255,256 |
| 数值范围  | Min-1, Min, Min+1<br>Max-1,Max,Max+1 | 输入数据允许1-999。<br><br>0, 1, 2, 998, 999, 1000               |
| 集合、空间 | 0, Min, Min+1<br>Max-1,Max,Max+1     | 图片大小不能超过50k<br><br>0, 1k, 49k, 50k, 51k                   |

# 5.测试用例（数据）确认方法

---

边界值附近数据的确认方法

**n:** 存在边界值的参数个数

**m:** 边界值条件数

**Paul Jorgensen公式:**

1.  **$4n+1$** : 基本边界测试
2.  **$6n+1$** : 健壮性边界测试
3.  **$3m$** : 边界条件测试

3种方法的测试力度依次增强

# 基本边界测试

## Paul Jorgensen公式

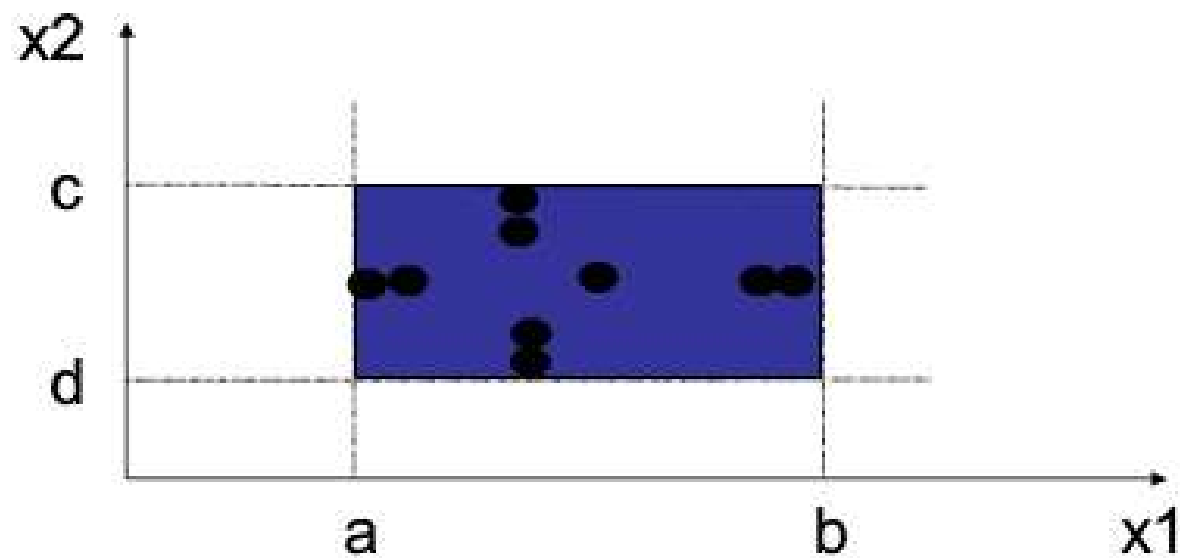
1.  $4n+1$ :  $n$ 为存在边界值的参数个数

每个参数取 $\min$ ,  $\min+1$ ,  $\max-1$ ,  $\max$ 各一次, 同时其他参数取典型值 $\text{nom}$ 。最后全部参数取典型值 $\text{nom}$ 一次。



- 例1: 有两个输入变量 $x_1(a \leq x_1 \leq b)$ 和 $x_2(c \leq x_2 \leq d)$ 的程序F的边界值分析测试用例如下:

{  $\langle x_1\text{nom}, x_2\text{min} \rangle$ ,  $\langle x_1\text{nom}, x_2\text{min}+ \rangle$ ,  $\langle x_1\text{nom}, x_2\text{nom} \rangle$ ,  
 $\langle x_1\text{nom}, x_2\text{max} \rangle$ ,  $\langle x_1\text{nom}, x_2\text{max}- \rangle$ ,  $\langle x_1\text{min}, x_2\text{nom} \rangle$ ,  
 $\langle x_1\text{min}+, x_2\text{nom} \rangle$ ,  $\langle x_1\text{max}, x_2\text{nom} \rangle$ ,  $\langle x_1\text{max}-, x_2\text{nom} \rangle$  }



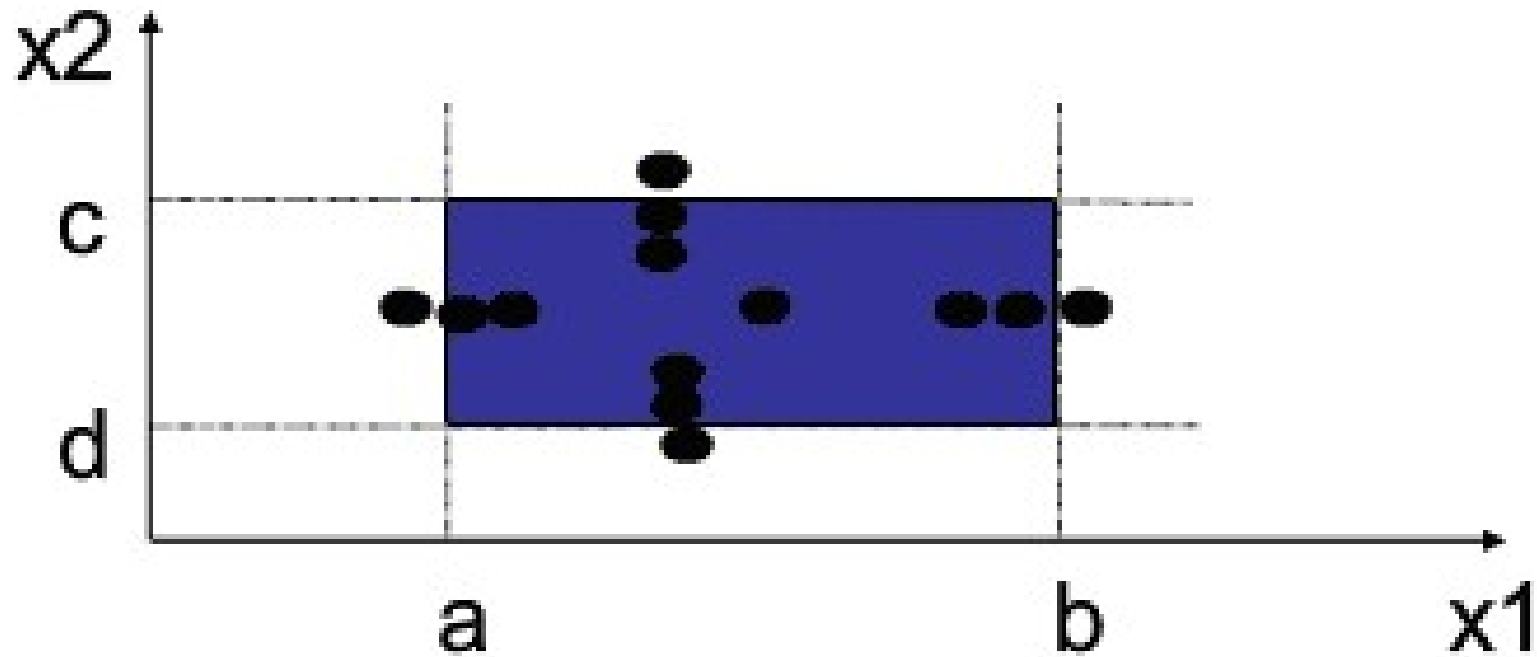
# 健壮性边界测试

Paul Jorgensen公式

2.  $6n+1$ :  $n$ 为存在边界值的参数个数  
【 $6*n*m+x?$ 】

每个参数取 $min-1$ ,  $min$ ,  $min+1$ ,  $max-1$ ,  $max$ ,  $max+1$ 各一次, 同时其他参数取典型值 $nom$ 。最后全部参数取典型值 $nom$ 一次。

前面例1中的测试用例按 $6n+1$ 公式扩充如下：



$\langle x1_{nom}, x2_{max+} \rangle, \langle x1_{nom}, x2_{min-} \rangle$

$\langle x1_{min-}, x2_{nom} \rangle, \langle x1_{max+}, x2_{nom} \rangle$

## C 边界值分析法...

---

### ■ 测试用例设计举例

| 大宗购买折扣         |      |
|----------------|------|
| 购买数量           | 单价   |
| 头10件（1-10件）    | 5.00 |
| 第二个10件（11-20件） | 4.75 |
| 第三个10件（21-30件） | 4.50 |
| 超过30件（31-∞）    | 4.00 |

15件需要支付： $10 \times 5 + 5 \times 4.75 = 73.75$

---

6n+1公式:  $6*3+3+1=22$ 个

边界1: 0、1、2

边界10: 9、10、11

边界11: 10、11、12

边界20: 19、20、21

边界21: 20、21、22

边界30: 29、30、31

边界31: 30、31、32

+X

## C 边界值分析法...

| 测试用例序号 | 测试的输入值 | 选择理由      | 预期输出  |
|--------|--------|-----------|-------|
| 1      | 0      | 第1阶段开始之前  | 0或报错  |
| 2      | 1      | 第1阶段开始/边界 | 5.00  |
| 3      | 2      | 第1阶段开始之后  | 10.00 |
| 4      | 9      | 第1阶段结束之前  |       |
| 5      | 10     | 第1阶段结束/边界 |       |
| 6      | 11     | 第1阶段结束之后  |       |
| 7      | 10     | 第2阶段开始前   |       |
| 8      | 11     | 第2阶段开始/边界 |       |
| 9      | 12     | 第2阶段开始后   |       |
| 10     | 19     | 第2阶段结束前   |       |
| 11     | 20     | 第2阶段结束/边界 |       |
| 12     | 21     | 第2阶段结束后   |       |
| 13     | 20     | 第3阶段开始前   | ...   |
| 14     | 21     | 第3阶段开始/边界 |       |



## C 边界值分析法...

---

### ■ 如何界定边界值

#### 问题描述:

**Next Date**程序需要输入3个整数，代表特定的月、日和年。同时，程序以月/日/年的格式返回下一日期。

#### **Special notes:**

计算1582年至3000年的日期



---

# 健壮性边界测试

## 1. 参数:

年 月 日      $n=3$

## 2. 确定边界值:

年: 1582, 3000

月: 1, 12

日: 1, 31

## 3. 测试用例设计:

方法二 ( $6n+1$ )

$18+x$ 个测试用例

| 测试用例 | 月    | 日    | 年         | 预期输出 |
|------|------|------|-----------|------|
| 1    | 0    | 1-31 | 1582-3000 | 错误   |
| 2    | 1    | 1-31 | 1582-3000 | 下一日期 |
| 3    | 2    | 1-31 | 1582-3000 | 下一日期 |
| 4    | 11   | 1-31 | 1582-3000 | 错误   |
| 5    | 12   | 1-31 | 1582-3000 | 下一日期 |
| 6    | 13   | 1-31 | 1582-3000 | 错误   |
| 7    | 1-12 | 0    | 1582-3000 | 错误   |
| 8    | 1-12 | 1    | 1582-3000 | 下一日期 |
| 9    | 1-12 | 2    | 1582-3000 | 下一日期 |
| 10   | 1-12 | 30   | 1582-3000 | 下一日期 |
| 11   | 1-12 | 31   | 1582-3000 | 下一日期 |
| 12   | 1-12 | 32   | 1582-3000 | 错误   |

| 测试用例 | 月    | 日    |           | 预期输出 |
|------|------|------|-----------|------|
| 13   | 1-12 | 1-31 | 1581      | 错误   |
| 14   | 1-12 | 1-31 | 1582      | 下一日期 |
| 15   | 1-12 | 1-31 | 1583      | 下一日期 |
| 16   | 1-12 | 1-31 | 2999      | 下一日期 |
| 17   | 1-12 | 1-31 | 3000      | 下一日期 |
| 18   | 1-12 | 1-31 | 3001      | 错误   |
| 19   | 1-12 | 1-31 | 1582-3000 | 下一日期 |
|      |      |      |           |      |
|      | ...  | ...  | ...       | ...  |

## C 边界值分析法…

---

边界条件测试

**Paul Jorgensen**公式

1. **3m**: m为边界条件数

每个条件取**-1**，自身，**+1**各一次。

---

# 边界条件测试

## 1.参数:

年 月 日  $n=3$

## 2.测试用例设计:

方法三 (3m)

3m个测试用例

## 3.确定边界条件:

(1) 每次只考虑一个参数的边界, 固定其它参数

(2) 补充确定的关联边界值

---

### 3.1 固定日、年的月边界条件

日：1-28

年：1582-3000

月：1, 12

| 边界条件 | 月  | 日    | 年         |
|------|----|------|-----------|
| 1    | 1  | 1-28 | 1582-3000 |
| 2    | 12 | 1-28 | 1582-3000 |

## 3.2 固定月、年的日边界条件

月：31/30天的月

年：1582-3000

日：1, 30, 31, 28, 29

| 边界条件 | 月     | 日  | 年         |
|------|-------|----|-----------|
| 3    | 30天的月 | 1  | 1582-3000 |
| 4    | 30天的月 | 30 | 1582-3000 |
| 5    | 31天的月 | 1  | 1582-3000 |
| 6    | 31天的月 | 31 | 1582-3000 |
| 9    | 2     | 1  | 闰年        |
| 10   | 2     | 29 | 闰年        |
| 11   | 2     | 1  | 非闰年       |
| 12   | 2     | 28 | 非闰年       |

### 3.3 固定月、日的年边界条件

月：1-12

日：1-31

年：1582，3000，闰年，非闰年

| 边界条件 | 月    | 日    | 年    |
|------|------|------|------|
| 7    | 1-12 | 1-31 | 1582 |
| 8    | 1-12 | 1-31 | 3000 |
| 9    | 2    | 1    | 闰年   |
| 10   | 2    | 29   | 闰年   |
| 11   | 2    | 1    | 非闰年  |
| 12   | 2    | 28   | 非闰年  |



### 3.4 补充确定的关联边界条件

| 边界条件 | 月  | 日  | 年    |
|------|----|----|------|
| 13   | 1  | 1  | 1582 |
| 14   | 12 | 31 | 3000 |

**m=14**

总的测试用例数:  $14 \times 3 = 42$

| 测试用例 | 边界条件 | 月    | 日    | 年         | 预期输出 |
|------|------|------|------|-----------|------|
| 1    | 1    | 0    | 1-28 | 1582-3000 | 错误   |
| 2    |      | 1    | 1-28 | 1582-3000 | 下一日期 |
| 3    |      | 2    | 1-28 | 1582-3000 | 下一日期 |
| 4    | 2    | 11   | 1-28 | 1582-3000 | 下一日期 |
| 5    |      | 12   | 1-28 | 1582-3000 | 下一日期 |
| 6    |      | 13   | 1-28 | 1582-3000 | 错误   |
| 7    | 3    | 31天月 | 0    | 1582-3000 | 错误   |
| 8    |      | 31天月 | 1    | 1582-3000 | 下一日期 |
| 9    |      | 31天月 | 2    | 1582-3000 | 下一日期 |
| ...  | ...  | ...  | ...  | ...       | ...  |

# 边界值分析测试练习——公历日期

用边界值分析方法设计合法公历日期的测试用例

日期范围：1582年1月1日—3000年12月31日

- ◆ 1582年10月5日至10月14日排除在公历外
- ◆ 2038年1月19日是BIOS提供的计时基准时间1970年1月1日的最大值(千年虫问题)
- ◆ 英国1752年才采用阳历，他们扣除9/3/1752到9/13/1752年同步以月亮为参照的立法

### 3.5 补充确定的关联边界条件

| 边界条件 | 月  | 日  | 年    |
|------|----|----|------|
| 15   | 1  | 19 | 2038 |
| 16   | 10 | 5  | 1582 |
| 17   | 10 | 14 | 1582 |
| 18   | 9  | 3  | 1752 |
| 19   | 9  | 13 | 1752 |

**m=19**

总的测试用例数： **$19 \times 3 = 57$**

## C 边界值分析...

---

### ■ 边界值分析和等价类划分的关系

等价类划分时，往往先要确定边界值。  
边界值分析是等价类划分方法的补充。  
测试中需要将两者结合起来使用。

# 讨 论

## □ 三角形程序的边值分析

| 测试用例   | a   | b   | c   | 预期输出  |
|--------|-----|-----|-----|-------|
| Test1  | 60  | 60  | 1   | 等腰三角形 |
| Test2  | 60  | 60  | 2   | 等腰三角形 |
| Test3  | 60  | 60  | 60  | 等边三角形 |
| Test4  | 50  | 50  | 99  | 等腰三角形 |
| Test5  | 50  | 50  | 100 | 非三角形  |
| Test6  | 60  | 1   | 60  | 等腰三角形 |
| Test7  | 60  | 2   | 60  | 等腰三角形 |
| Test8  | 50  | 99  | 50  | 等腰三角形 |
| Test9  | 50  | 100 | 50  | 非三角形  |
| Test10 | 1   | 60  | 60  | 等腰三角形 |
| Test11 | 2   | 60  | 60  | 等腰三角形 |
| Test12 | 99  | 50  | 50  | 等腰三角形 |
| Test13 | 100 | 50  | 50  | 非三角形  |

# 第四次作业(2024.4.18)

---

- 1.如何用等价类划分方法设计公历日期程序的测试用例？
- 2.如何结合等价类划分和边界值分析设计公历日期程序的测试用例？